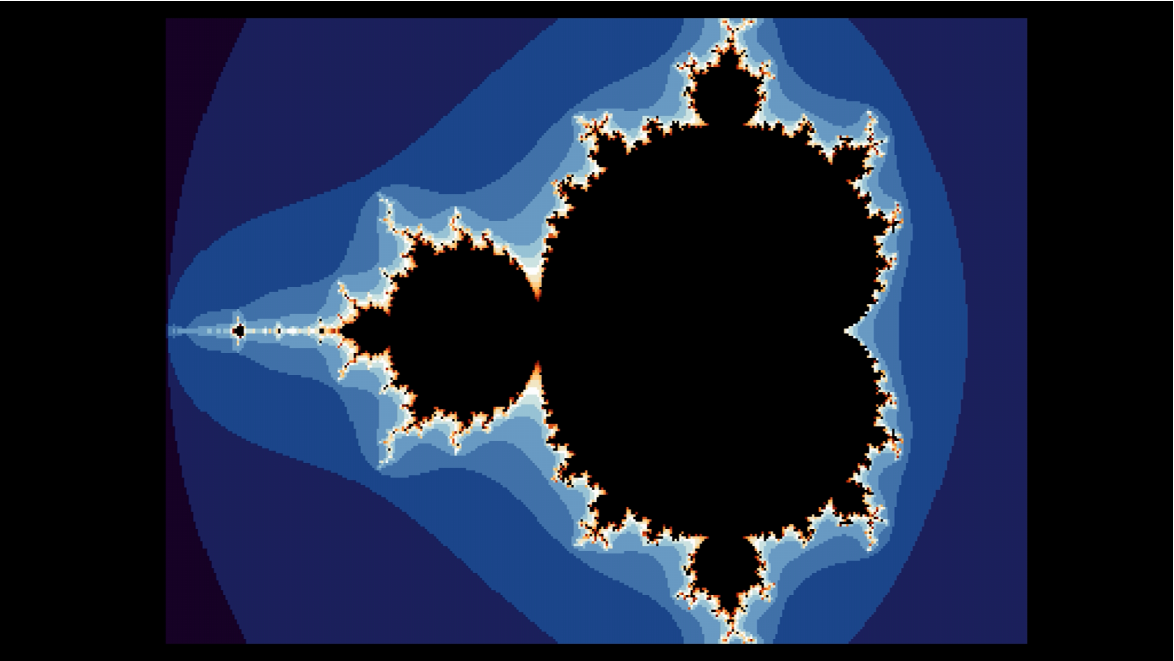


Contents

Mandelbrot Upic	1
Contents	1
Controls	2
Installation	5
Building from source	6
Documentation	7

Mandelbrot Upic



An Ultimate 64 demo that generates a Mandelbrot fractal on-device at 64 MHz turbo, packs it directly into Upic format (a 16-color, 384x256 border-color raster picture technique), displays it live as it renders, and lets you interactively pan around and zoom into any region of the result.

The same PRG also runs on an original Ultimate 64 or Ultimate 64 Elite (48 MHz turbo maximum). It measures the CPU speed at startup and, on a 48 MHz machine, shows 288 of the 384 pixels per line (3 of every 4, each about 1.36 dots wide) at the same size and screen position: every shown pixel is within 1.5 dots of where a 64 MHz machine shows it. The 48 MHz path has been checked on an Ultimate 64 Elite, against a C64 Ultimate running the same PRG (see make e2e under [Make targets](#)).

See CREDITS.md for full attribution.

Status: v1.1.0, feature-complete and confirmed working on real hardware: an Ultimate 64 Elite II (64 MHz path) and an Ultimate 64 Elite (48 MHz path), both on firmware 3.15a.

Watch it in action -- real-hardware capture: live generation, all 4 palettes, and interactive zoom.

Contents

- [Controls](#)
- [Installation](#)

- [Building from source](#)
- [Documentation](#)
- [Changelog](#)
- [Credits](#)

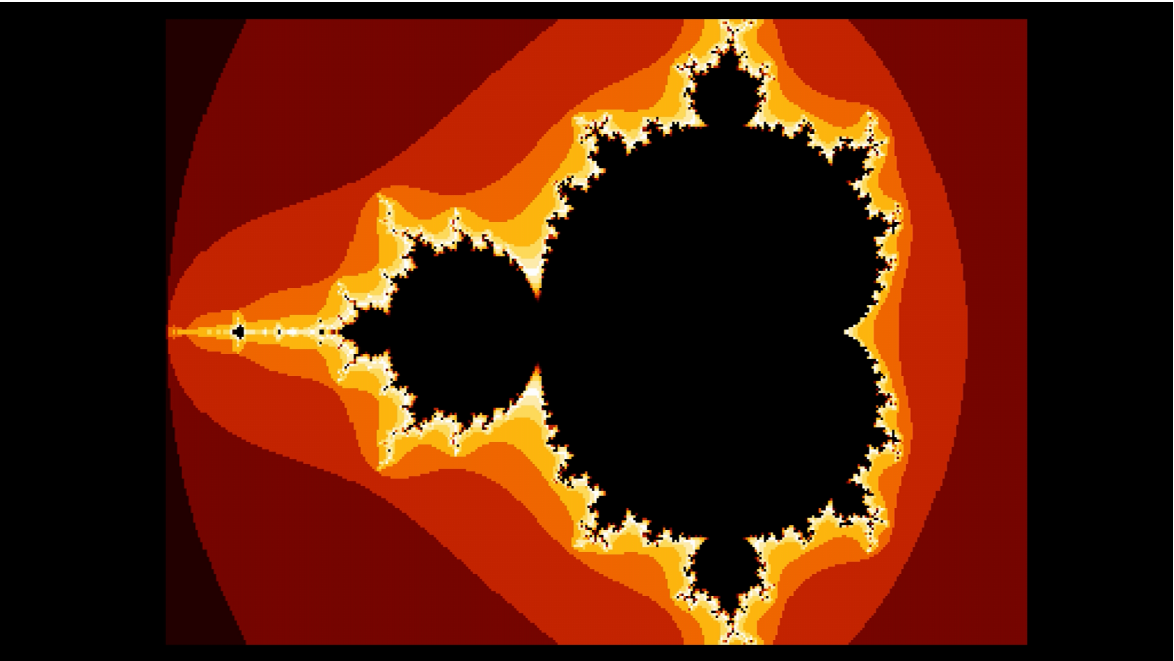
Controls

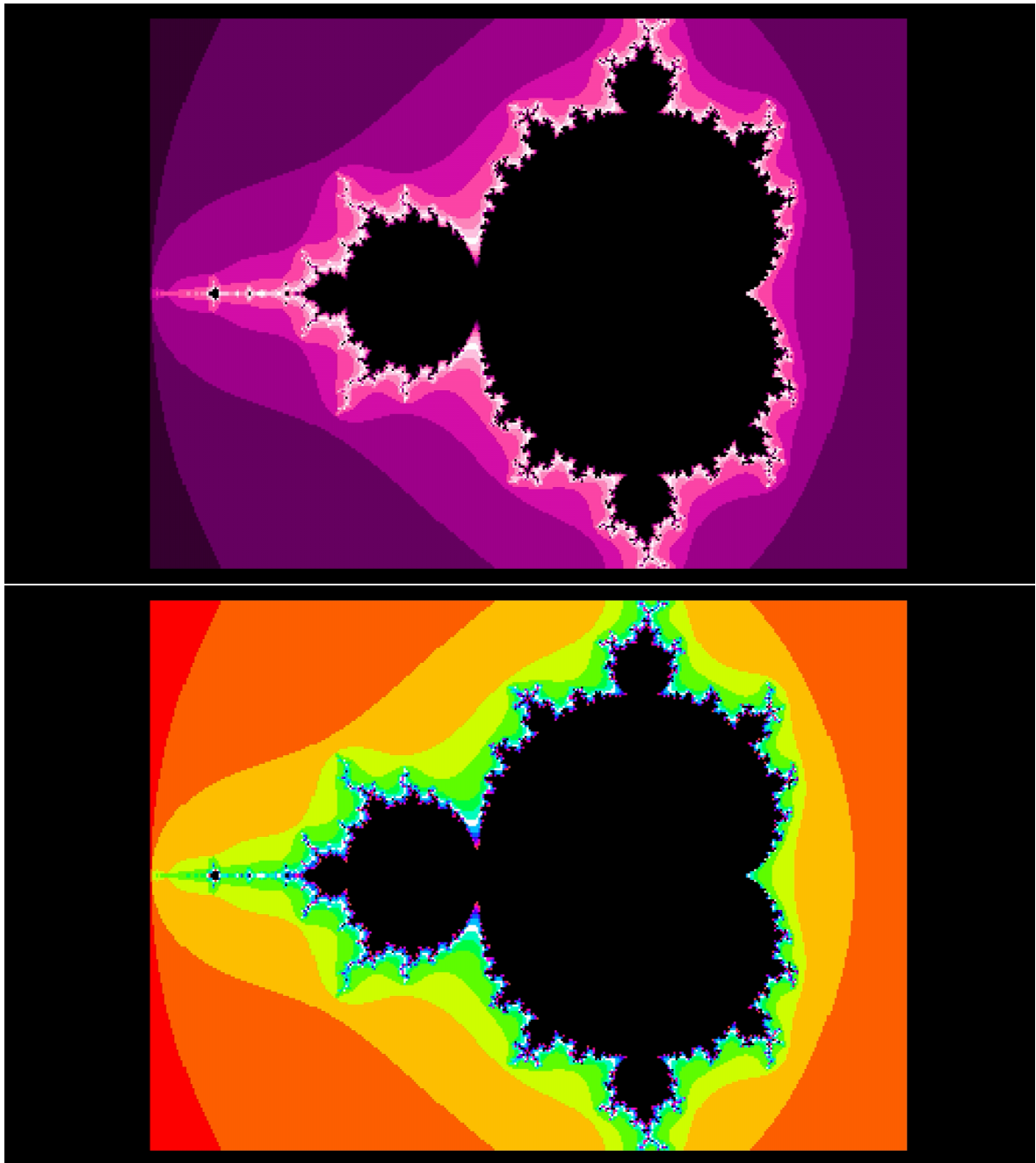
Generation starts automatically on launch and the picture builds up live, left to right. Once it completes, you're in **browse mode**:

Input	Action
W/A/S/D	Pan the current view (no-op at the default overview -- nothing to pan to)
Cursor keys (hold Shift for up/left)	Same, alternative for muscle memory
Z	Enter box mode to pick a zoom target
O	Zoom out one notch (widens the view, clamped to the original overview)
C	Cycle the base color gradient (sunset, fire, amethyst, rainbow)

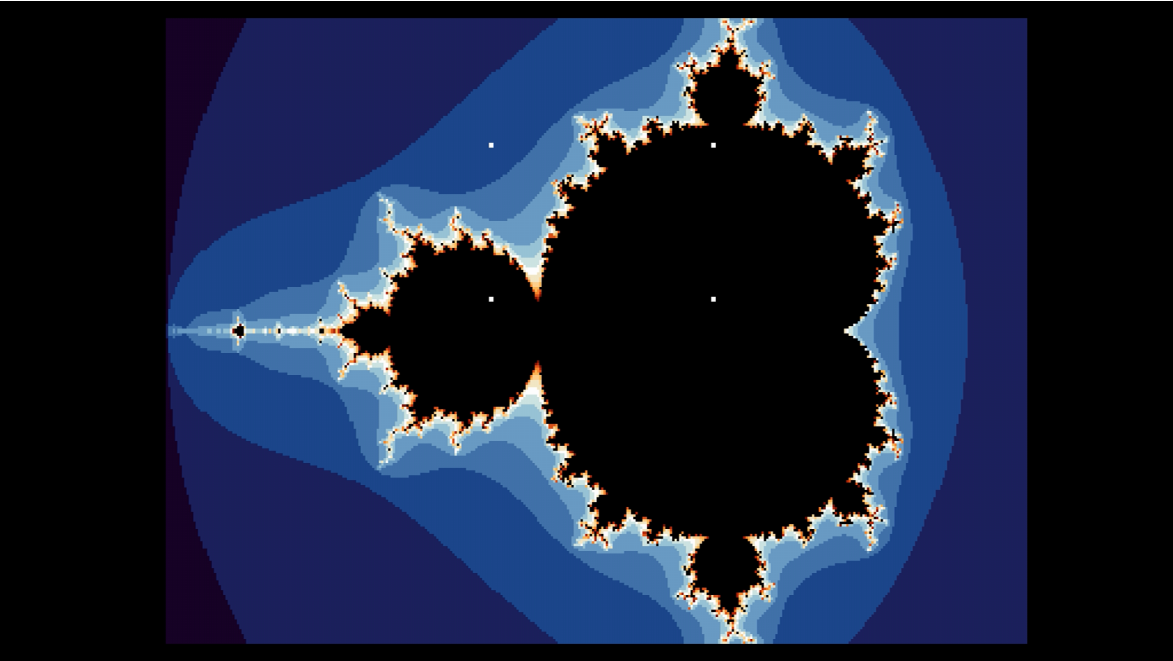
Panning regenerates the fractal at the same zoom level, shifted -- each step is a fraction of the current view's own size, so it moves less in absolute terms the deeper you've zoomed in.

The 4 selectable gradients:



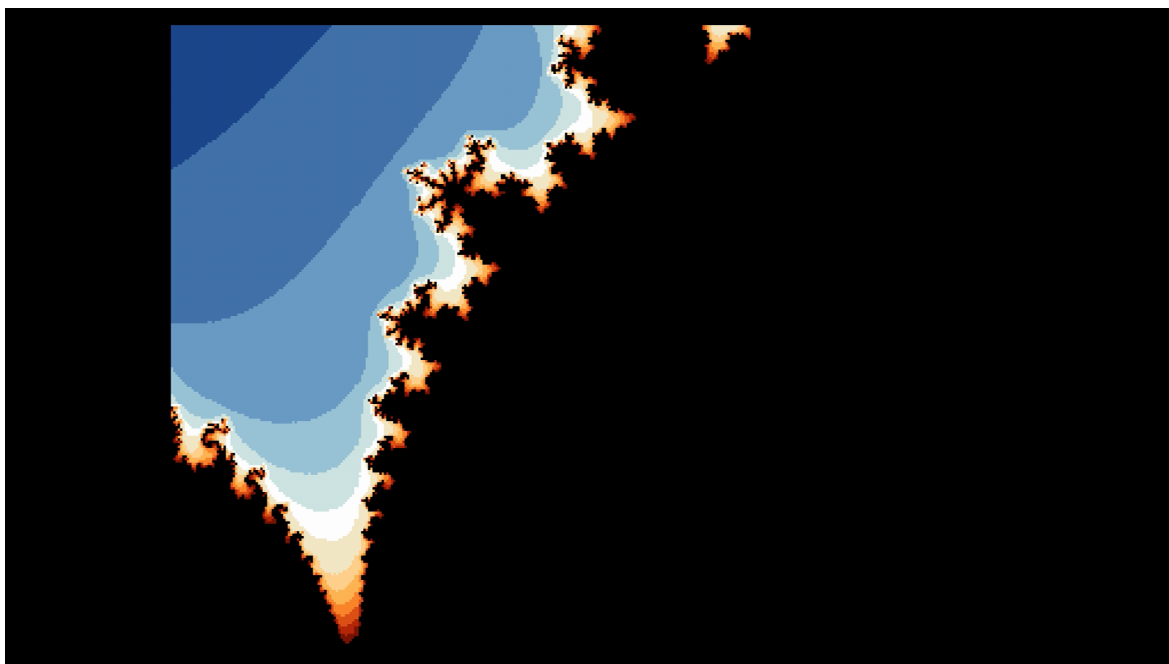


Press Z to enter **box mode**: 4 corner markers (solid white 2x2 blocks) appear, outlining a box that always keeps the picture's own 3:2 aspect ratio:



Input	Action
W/A/S/D	Move the whole box
Cursor keys (hold Shift for up/left)	Same, alternative for muscle memory
+	Grow the box (aspect ratio unchanged)
-	Shrink the box (aspect ratio unchanged)
RETURN	Confirm and zoom into the box
Z	Cancel back to browse mode without zooming
C / O	Same as browse mode

Confirming regenerates the fractal at the selected region and returns to browse mode. Repeated zooms compose relative to whatever's currently displayed, so zooming, panning, and zooming again all work together.



Zoom precision limit: the fractal coordinates use a fixed-point format with a finite number of fractional bits, capping how far repeated zooming can go (roughly 16x total from the initial view) before individual pixel steps round down to zero and the picture stops changing with further zoom. This is a hard limit of the current implementation, not a bug -- see docs/MANDELBROT_ALGORITHM.md.

No quit key -- reset or power off to exit, same as many C64 demos with no graceful exit path. See docs/ZOOM_FEATURE.md for why.

Installation

Requires **firmware 3.15 or newer** on an **Ultimate 64**, **Ultimate 64 Elite** or **Ultimate 64 Elite II**. The Elite II runs at 64 MHz and shows the full picture; the original Ultimate 64 and the Elite top out at 48 MHz and use the 48 MHz display path described above. The Commodore 64 Ultimate (C64U), built on the Elite II design with its own firmware branch, needs the C64U firmware release that adds palette control (expected to be 1.2, not yet released at the time of writing); the same PRG already ran on a 1.2 release candidate during the 48 MHz work's testing.

1. Copy both `mandelupic.prg` and `mandelupic.cfg` onto your Ultimate's SD card or USB storage, in the same folder -- extracting the release ZIP already places them together (at `idi8b/mandelupic/`), or `make deploy/make zip` produce them from source with matching names.
2. Run `mandelupic.prg` from the Ultimate's file browser.

The Ultimate's own firmware auto-loads a config file that shares its base name with the program being run -- `mandelupic.cfg` next to `mandelupic.prg` is picked up automatically, no manual "load config" step needed. It enables the Command Interface (UCI) and the turbo registers this demo needs; if your own configuration already has both enabled, this has no effect either way.

The same `mandelupic.cfg` works on every supported machine, so there is nothing to choose or rename. The turbo setting has a different value name per product (U64 Turbo Registers on an Ultimate 64, C64U Turbo Registers on a C64 Ultimate), so the file contains both lines: the firmware

skips the value it doesn't know and applies the one it does, silently when the file is auto-loaded. (Loading the file by hand from the menu shows a brief message about the skipped line; that is expected and harmless.) If the demo still doesn't run at turbo speed, set Turbo Control to your machine's "... Turbo Registers" value in the Ultimate menu.

Building from source

Prerequisites

Tool	Purpose	Install
Oscar64	C cross-compiler targeting 6502/C64	build from source, see their README
wput	FTP deploy to the Ultimate device	<code>sudo apt install wput</code>
pandoc + texlive-xetex	Generate README.pdf (optional)	<code>sudo apt install pandoc texlive-xetex</code>
Python 3	make test, make e2e (optional)	usually preinstalled

Deploy setup

Copy `.env.example` to `.env` and set `ULTIP1` to your Ultimate device's IP address (and, for make e2e, `E2E_DEVICES` to the devices to test on):

```
cp .env.example .env
```

Make targets

Target	Effect
<code>make / make all</code>	Compile to <code>build/mandelupic.prg</code> , regenerate <code>README.pdf</code> , build the release ZIP
<code>make deploy</code>	FTP the compiled <code>.prg</code> and matching <code>.cfg</code> to the Ultimate device set in <code>.env</code>
<code>make test</code>	Build both PRGs and run the host-side tests in <code>tests/</code> (Python 3, standard library only)
<code>make force48</code>	Build the test-only <code>build/mandelupic-force48.prg</code> , which always uses the 48 MHz display path (at 48 MHz), so that path can be checked on an Elite II / C64U
<code>make deploy-force48</code>	FTP the force48 PRG (as <code>mandelupic-force48.prg</code> , with its own copy of the <code>.cfg</code>) next to the release one

Target	Effect
<code>make e2e</code>	Run the release PRG on the real devices listed in <code>E2E_DEVICES</code> in <code>.env</code> , drive it with key presses over the REST API, and compare the VIC video stream with the golden images in <code>tests/e2e/golden/</code> (see <code>tests/e2e/README.md</code>)
<code>make e2e-update</code>	Same run, but write the captures as the new golden images for each device's display path
<code>make docs</code>	Regenerate <code>README.pdf</code> via <code>pandoc</code>
<code>make clean</code>	Remove build outputs

Documentation

Document	Covers
docs/ARCHITECTURE.md	Program flow, components, memory layout overview
docs/MANDELBROT_ALGORITHM.md	Fixed-point fractal generation algorithm
docs/UPIC_VIEWER.md	The Upic border-color raster display technique
docs/ZOOM_FEATURE.md	Interactive pan/zoom/palette control scheme
docs/TURBOCONTROL_MANUAL.md	Ultimate 64 CPU speed control library
docs/UCILIB_MANUAL.md	Ultimate Command Interface (UCI) protocol library
docs/OSCAR64_MANUAL.md	Oscar64 compiler reference (copy of the maintainer's canonical manual)
tests/README.md / tests/e2e/README.md	Host-side timing tests and the end-to-end test on real hardware
CHANGELOG.md	Version history
CREDITS.md	Full attribution